theory *Week_2_sol*
imports *Main*
begin

King's College London
Department of Informatics
Dr. Mohammad Abdulaziz

Term 1

Computer-Assisted Theorem Proving: 7CCSACTP

Exercise Sheet 2

About these solutions. Every Isabelle proof method (e.g. *rule*, *simp*), Isar keyword (e.g. *have*, *also*), attribute (e.g. *OF*, *of*) and method modifier (e.g. *arbitrary:*, *simp add:*) is explained in a short comment, marked —, the first time it appears in the solutions, in sheet order from Week 1. Later uses are not explained again, so if you meet an unfamiliar construct, look for its first use in an earlier solution. Each proof is also labelled with its style; the module's preferred style is declarative (Isar), where every step states what it proves.

Important Note: Please download Isabelle from the link provided in the slides and bring a laptop to the large group tutorial. Performing the proofs in Isabelle will be an integral part of those tutorials.

Before you start, please open a new Isabelle/HOL theory file called Week_1.thy and add the following to its beginning.

theory Week_2
imports Main
begin

Exercise 2.1 Addition is Associative

Consider the function (+) defined in Isabelle/HOL for natural numbers.

Prove the following theorem, first pen-and-paper, and then formally in Isabelle.

lemma *add_associative*: $((a::nat) + b) + c = a + (b + c)$

Hints: 1. Perform the proof by induction, using substitutions *subst*.

2. Think about which variable to perform induction on. Addition on natural numbers is defined by recursion on its first argument: look at *thm add_0_left add_Suc*.

3. You will need to identify multiple lemmas about how *add* works. Do not prove those lemmas. Use *sorry* as the proof. This is the top-down approach.

4. There is a proof which uses only the following lemmas: *add_0_left*, *add_Suc*

proof(*induction a*)
— *induction a* splits the goal into a base case *0* and a step case *Suc a*, whose induction hypothesis is *Suc.IH*; each case is entered with *case*.
— Style: declarative (Isar); induction (new); equational reasoning (*also*/*finally*); *subst*.
case *0*
— *case 0* enters the base case; *?case* abbreviates its goal, and *next* moves on to the next case.
have *i*: $(0 + b) + c = b + c$
apply(*subst add_0_left*) **..**
— .. finishes the goal by applying a standard rule; here *refl* closes $b + c = b + c$.
also have ... $= 0 + (b + c)$
— *also* chains this step to the previous one, with ... standing for its right-hand side; *finally* combines the whole chain into one fact.
apply(*subst add_0_left*) **..**
finally show $(0 + b) + c = 0 + (b + c)$
.
next
case (*Suc a*)
thm *Suc.IH*
have *i*: $(Suc\ a + b) + c = Suc\ (a + b) + c$
apply(*subst add_Suc*) **..**
also have ... $= Suc\ ((a + b) + c)$
apply(*subst add_Suc*) **..**
also have ... $= Suc\ (a + (b + c))$
apply(*subst Suc.IH*) **..**
also have ... $= Suc\ a + (b + c)$ **thm** *add_Suc*[*symmetric*]
apply(*subst add_Suc*[*symmetric*]) **..**
— *th*[*symmetric*] swaps the two sides of the equation *th*, so *subst* rewrites right to left; here $Suc\ (a + (b + c))$ becomes $Suc\ a + (b + c)$.
finally show *?case* **.**
qed

Exercise 2.2 Truncated Subtraction

Subtraction on natural numbers is truncated: $n - m = 0$ whenever $n \leq m$. Prove that, as long as $m \leq n$, subtracting m and adding it back gives n again. You should prove it both, pen-and-paper and in Isabelle, by induction on n.

lemma *sub_add*: $(m::nat) \leq n \Longrightarrow (n - m) + m = n$

Hints: 1. In the step case, perform a case analysis with *proof*(*cases* $m \leq n$). In the second case, together with the assumption $m \leq Suc\ n$, you can derive $m = Suc\ n$.

2. Use *thm Suc.IH* in the step case to look at the induction hypothesis. It has an assumption, which you can discharge with *OF*.

3. The following lemmas suffice: *le0*, *le_antisym*, *diff_self_eq_0*, *add_0_left*, *Suc_diff_le*, *add_Suc*, *not_le_imp_less*, *Suc_leI*.

proof(*induction n*)
— Style: declarative (Isar); induction for a lemma with an assumption (new); case analysis on a formula with *cases* (new) in the step case; equational reasoning; *OF*; *subst*.
case *0*
have *1*: $m = 0$
using *le_antisym*[*OF 0 le0*]
.
have $0 - m + m = 0 - 0 + m$
apply(*subst 1*)
..
also have ... $= 0 + m$
apply(*subst diff_self_eq_0*)
..
also have ... $= m$
apply(*subst add_0_left*)
..
also have ... $= 0$
using *1*
.
finally show *?case*
.
next
case (*Suc n*)
show *?case*
proof(*cases* $m \leq n$)
— For a formula P, *cases P* gives the cases *True*, assuming P, and *False*, assuming $\neg P$; the assumption is available under the case name.
case *True*
have *Suc n* $-$ $m + m =$ *Suc* $(n - m) + m$
apply(*subst Suc_diff_le*[*OF True*])
..
also have ... = *Suc* $(n - m + m)$
apply(*subst add_Suc*)
..
also have ... = *Suc n*
apply(*subst Suc.IH*[*OF True*])
..
finally show *?thesis*
.
next
case *False*
have *m_eq*: $m =$ *Suc n*
using *le_antisym*[*OF Suc.prems Suc_leI*[*OF not_le_imp_less*[*OF False*]]]
— *Suc.prems* are the assumptions of the current case, here $m \leq$ *Suc n*.

```
    .
  have Suc n − m + m = Suc n − Suc n + m
    apply(subst m_eq)
    ..
  also have ... = 0 + m
    apply(subst diff_self_eq_0)
    ..
  also have ... = m
    apply(subst add_0_left)
    ..
  also have ... = Suc n
    using m_eq
    .
  finally show ?thesis
    .
 qed
qed
```

Exercise 2.3 Induction Hypothesis Generalisation with an Accumulator

Consider the following function, which adds two natural numbers using an accumulator: in each recursive call it moves one *Suc* from its first argument to its second.

```
fun add_acc :: nat ⇒ nat ⇒ nat where
  add_acc 0 m = m
| add_acc (Suc n) m = add_acc n (Suc m)
```

We want to prove that *add_acc* computes addition.

```
lemma add_acc_correct: add_acc n m = n + m
```

(a) First try to prove the lemma by plain induction on *n*, i.e. with *proof*(*induction n*). Write down the goal of the step case and the induction hypothesis. Explain, pen-and-paper, why the induction hypothesis is not strong enough to prove the step case.

(b) Now prove the lemma, first pen-and-paper and then formally in Isabelle, by generalising the induction hypothesis over *m*, i.e. with *proof*(*induction n arbitrary*: *m*). Your pen-and-paper proof should state the generalised induction hypothesis and say with which term you instantiate its universally quantified variable in the step case.

Hints: 1. *add_acc* is a recursive function defined with *fun*, like *add* in Section 2.2.2 of Concrete Semantics. Its defining equations are available as *add_acc.simps*(*1*) and *add_acc.simps*(*2*) (look at *thm add_acc.simps*), and can be used with *subst* like any other equation.

2. Use *thm Suc.IH* in the step case to compare the induction hypothesis you get with and without *arbitrary*: *m*.

3. The following lemmas suffice: *add_acc.simps*, *add_0_left*, *add_Suc*, *add_Suc_right*.

Solution (a). With *proof*(*induction n*), the step case has to show *add_acc* (*Suc n*) *m* = *Suc n* + *m*, i.e. after unfolding *add_acc*, *add_acc n* (*Suc m*) = *Suc n* + *m*. The induction hypothesis is *add_acc n m* = *n* + *m* for the same, fixed *m*. It talks about *add_acc n m*, but the goal is about *add_acc n* (*Suc m*), so the induction hypothesis cannot be applied: the second argument changes in the recursive call.

Solution (b). With *arbitrary*: *m* the induction hypothesis becomes ⋀*m*. *add_acc n m* = *n* + *m*. In the step case we instantiate it with *Suc m*, giving *add_acc n* (*Suc m*) = *n* + *Suc m* = *Suc* (*n* + *m*) = *Suc n* + *m*.

proof(*induction n arbitrary*: *m*)
— *arbitrary*: *m* generalises *m*, so the induction hypothesis holds for every *m*.
— Style: declarative (Isar); generalised induction with *arbitrary* (new) for a recursive function; IH instantiated with *of* (new); equational reasoning.
case *0*
have *add_acc 0 m* = *m*
apply(*subst add_acc.simps*(*1*))
..
also have ... = *0* + *m*
apply(*subst add_0_left*)
..
finally show *?case*
.
next
case (*Suc n*)
have *add_acc* (*Suc n*) *m* = *add_acc n* (*Suc m*)
apply(*subst add_acc.simps*(*2*))
..
also have ... = *n* + *Suc m*
apply(*subst Suc.IH*[*of Suc m*])
— *th*[*of t*] instantiates the first variable of *th* with the term *t*; here the IH with *Suc m* for *m*.
..
also have ... = *Suc* (*n* + *m*)
apply(*subst add_Suc_right*)
..
also have ... = *Suc n* + *m*
apply(*subst add_Suc*)
..
finally show *?case*
.
qed

(c) Challenge (optional) The following function multiplies using an accumulator. Prove, pen-and-paper and in Isabelle, that it computes *acc* + *n* ∗ *m*. Which variable must you generalise over this time, and why?

```
fun mult_acc :: nat ⇒ nat ⇒ nat ⇒ nat where
  mult_acc 0 m acc = acc
| mult_acc (Suc n) m acc = mult_acc n m (acc + m)

lemma mult_acc_correct: mult_acc n m acc = acc + n * m
```

Hint: The following lemmas suffice: *mult_acc.simps*, *mult_0*, *add_0_right*, *mult_Suc*, *add.assoc*.

Solution (c). We generalise over *acc*: the recursive call changes the accumulator from *acc* to *acc* + *m*, so the induction hypothesis is needed for *acc* + *m*. The argument *m* stays the same in the recursive call, so it does not need to be generalised.

```
proof(induction n arbitrary: acc)
  — Style: as in (b).
  case 0
  have mult_acc 0 m acc = acc
    apply(subst mult_acc.simps(1))
    ..
  also have ... = acc + 0
    apply(subst add_0_right)
    ..
  also have ... = acc + 0 * m
    apply(subst mult_0)
    ..
  finally show ?case
    .
next
  case (Suc n)
  have mult_acc (Suc n) m acc = mult_acc n m (acc + m)
    apply(subst mult_acc.simps(2))
    ..
  also have ... = (acc + m) + n * m
    apply(subst Suc.IH)
    ..
  also have ... = acc + (m + n * m)
    apply(subst add.assoc)
    ..
  also have ... = acc + Suc n * m
    apply(subst mult_Suc)
    ..
  finally show ?case
    .
qed
```

end